

Produktiveres Data Engineering mit SAS Viya Copilot im SAS Data & AI Studio

Plattform Netzwerktreffen

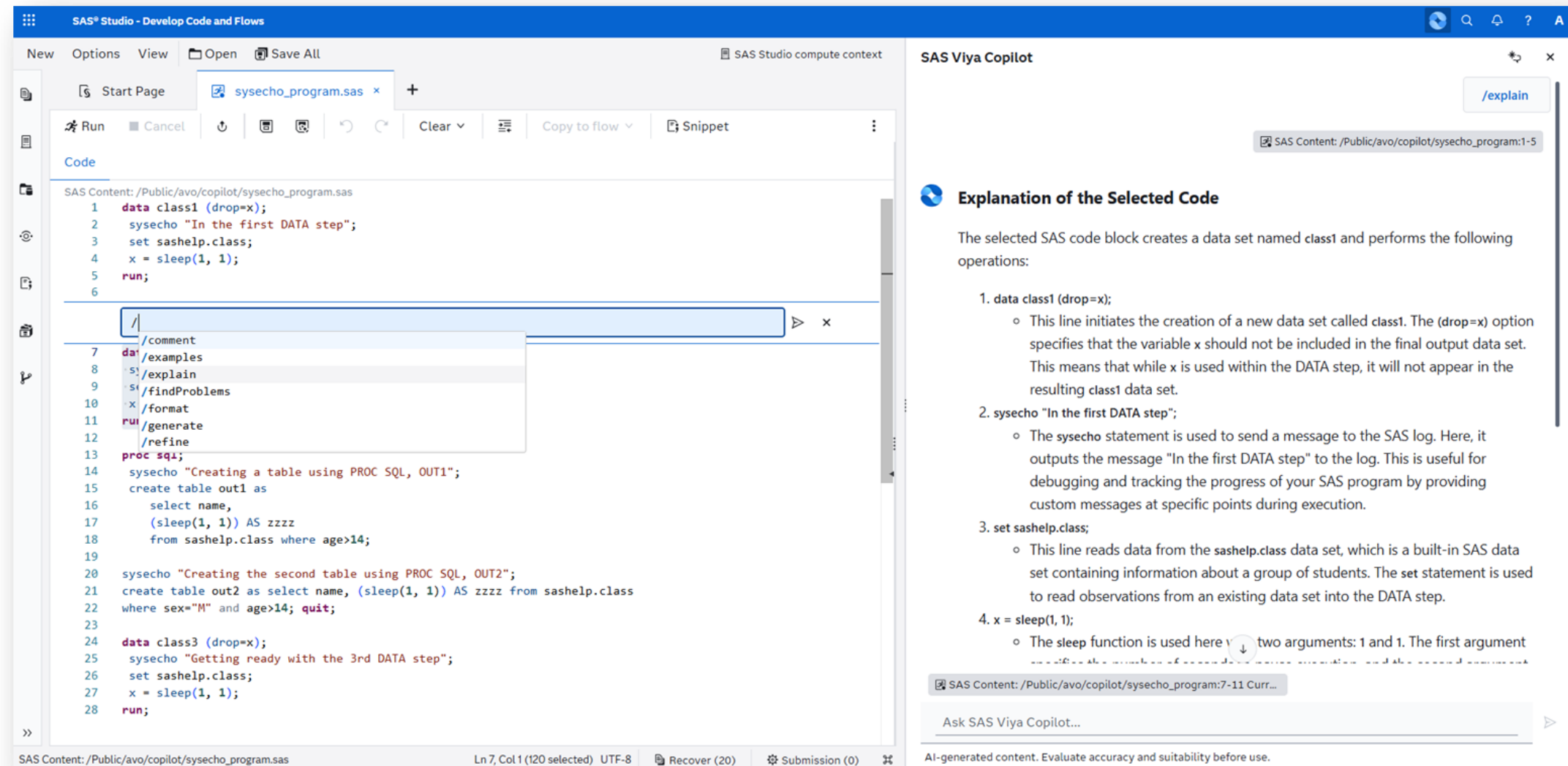
20. Mai 2026



Agenda

- 1 Welches Release enthält den Copiloten?
- 2 Funktionen des Code Assistant
- 3 Funktionen des Flow Assistant (Roadmap)
- 4 Q&A

Welches Release enthält den Copiloten?



18. Juni
2026.06 Stable Release

Umfang des ersten SAS Studio & AI Release?



Included

- Dokumentation Q&A
- Code Assistant für SAS Code
- Code Assistant für Python Code

Comes next

- Code Assistant für R Code
- Flow Assistant: Steps Generation
- Flow Assistant: Step Description Generation
- Flow Assistant: Flow Dokumentation
- und mehr



Heute schauen wir uns an (Demo)



Included

- Dokumentation Q&A
- Code Assistant für SAS Code
- Code Assistant für Python Code

Comes next

- Code Assistant für R Code
- Flow Assistant: Steps Generation
- Flow Assistant: Step Description Generation
- Flow Assistant: Flow Dokumentation
- und mehr



Initial release

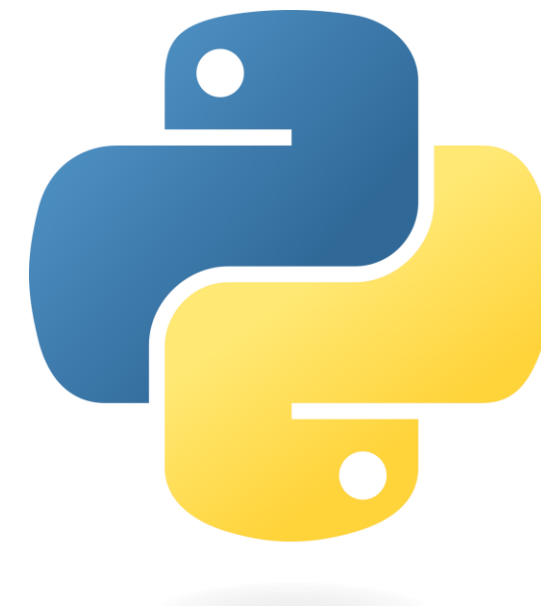
Code Assistant

Für SAS und Python Code

Überblick

Eine Reihe von Aktionen zur Erstellung neuen Codes und zur Arbeit mit bestehendem Code, verfügbar über den Copilot-Chatbot-Panel und Inline im Programmcode

Sprachenunterstützung



Aktionen

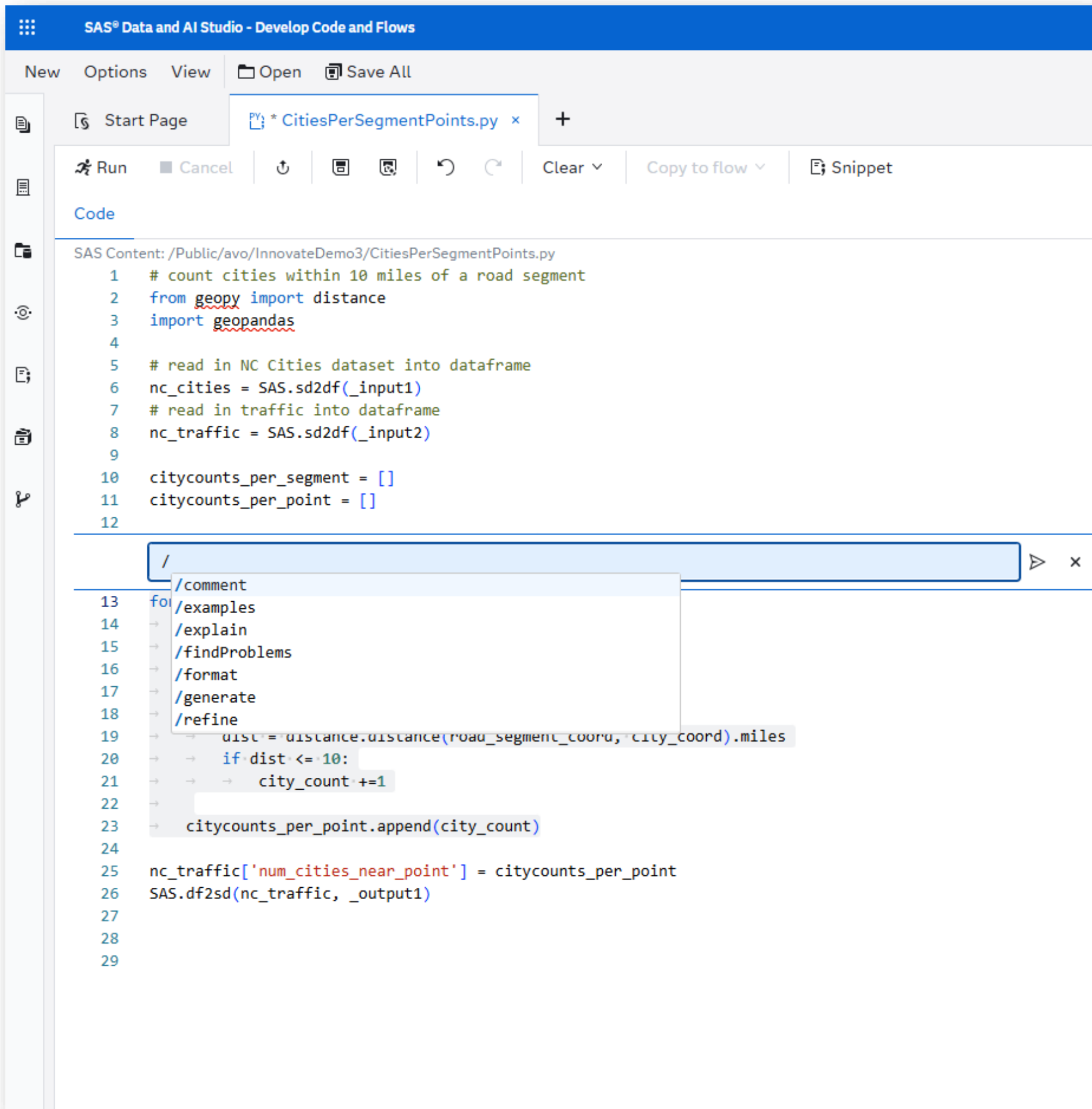
Für neuen Code

- ✓ Generate
- ✓ Examples

Für bestehenden Code

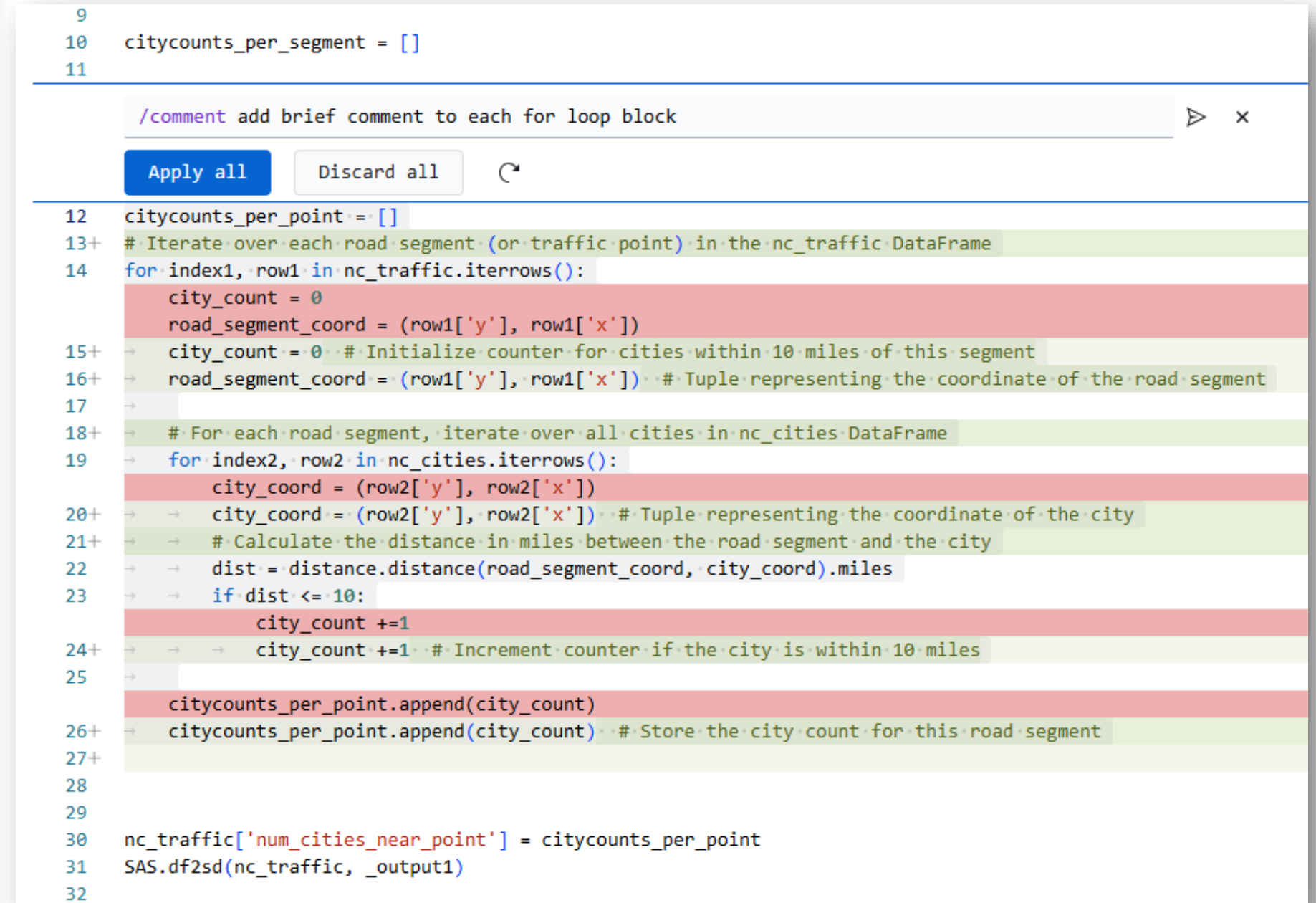
- ✓ Comment
- ✓ Explain
- ✓ Find problems
- ✓ Format
- ✓ Refine

Nutzung Inline



The screenshot shows the SAS Data and AI Studio interface. The top bar is blue with the text "SAS® Data and AI Studio - Develop Code and Flows". Below the bar is a menu bar with "New", "Options", "View", "Open", and "Save All". The main area is divided into a "Code" tab and a "Run" tab. The "Code" tab is active, showing a Python script named "CitiesPerSegmentPoints.py". The script is a Jupyter-style notebook with code cells and a comment menu. The comment menu is open, showing options like "/comment", "/examples", "/explain", "/findProblems", "/format", "/generate", and "/refine". The script content is as follows:

```
SAS Content: /Public/avo/InnovateDemo3/CitiesPerSegmentPoints.py
1 # count cities within 10 miles of a road segment
2 from geopy import distance
3 import geopandas
4
5 # read in NC Cities dataset into dataframe
6 nc_cities = SAS.sd2df(_input1)
7 # read in traffic into dataframe
8 nc_traffic = SAS.sd2df(_input2)
9
10 citycounts_per_segment = []
11 citycounts_per_point = []
12
13 for
14 /comment
15 /examples
16 /explain
17 /findProblems
18 /format
19 /generate
20 /refine
21 dist = distance.distance(road_segment_coord, city_coord).miles
22 if dist <= 10:
23     city_count += 1
24 citycounts_per_point.append(city_count)
25
26 nc_traffic['num_cities_near_point'] = citycounts_per_point
27 SAS.df2sd(nc_traffic, _output1)
28
29
```



The screenshot shows a code editor with a Python script. The script is a Jupyter-style notebook with code cells and a comment menu. The comment menu is open, showing options like "/comment", "/examples", "/explain", "/findProblems", "/format", "/generate", and "/refine". The script content is as follows:

```
9
10 citycounts_per_segment = []
11
12 /comment add brief comment to each for loop block
13
14 Apply all Discard all
15
16 citycounts_per_point = []
17 # Iterate over each road segment (or traffic point) in the nc_traffic DataFrame
18 for index1, row1 in nc_traffic.iterrows():
19     city_count = 0
20     road_segment_coord = (row1['y'], row1['x'])
21     city_count = 0 # Initialize counter for cities within 10 miles of this segment
22     road_segment_coord = (row1['y'], row1['x']) # Tuple representing the coordinate of the road segment
23
24     # For each road segment, iterate over all cities in nc_cities DataFrame
25     for index2, row2 in nc_cities.iterrows():
26         city_coord = (row2['y'], row2['x'])
27         city_coord = (row2['y'], row2['x']) # Tuple representing the coordinate of the city
28         # Calculate the distance in miles between the road segment and the city
29         dist = distance.distance(road_segment_coord, city_coord).miles
30         if dist <= 10:
31             city_count += 1
32             city_count += 1 # Increment counter if the city is within 10 miles
33
34     citycounts_per_point.append(city_count)
35 citycounts_per_point.append(city_count) # Store the city count for this road segment
36
37 nc_traffic['num_cities_near_point'] = citycounts_per_point
38 SAS.df2sd(nc_traffic, _output1)
39
```

SAS Data and AI Studio compute context

Py * CitiesPerSegmentPoints.py x

Run Cancel

Code

SAS Content: /Public/avo/InnovateDemo3/CitiesPerSegmentPoints.py

```
1 # count cities within 10 miles of a road segment
2 from geopy import distance
3 import geopandas
4
5 # read in NC Cities dataset into dataframe
6 nc_cities = SAS.sd2df(_input1)
7 # read in traffic into dataframe
8 nc_traffic = SAS.sd2df(_input2)
9
10 citycounts_per_segment = []
11 citycounts_per_point = []
12
13 for index1, row1 in nc_traffic.iterrows():
14     city_count = 0
15     road_segment_coord = (row1['y'], row1['x'])
16
17     for index2, row2 in nc_cities.iterrows():
18         city_coord = (row2['y'], row2['x'])
19         dist = distance.distance(road_segment_coord, city_coord).miles
20         if dist <= 10:
21             city_count +=1
22
23     citycounts_per_point.append(city_count)
24
25 nc_traffic['num_cities_near_point'] = citycounts_per_point
26 SAS.df2sd(nc_traffic, _output1)
27
28
```

SAS Viya Copilot

add one extensive comment to outer for loop block, don't add comments within the block

Py SAS Content: /Public/avo/InnovateDemo3/CitiesPerSegmen...

Certainly! Below is your code with an extensive comment added to the **outer for loop block** (the loop iterating over road segments in `nc_traffic`). As requested, there are **no comments within the block itself**; the comment is placed directly above the loop and fully explains its purpose and logic.

```
# The following outer loop iterates over each road segment
# For every segment, it initializes a counter to track the number of cities
# It then compares the coordinates of the segment to the coordinates of each city
# using geopy's distance calculation. If a city is found within 10 miles, the counter is incremented.
# The final count for each segment is appended to the citycounts_per_point list.
# This process effectively builds a list where each element represents the number of cities
# enabling spatial analysis such as identifying segments with high city density.
for index1, row1 in nc_traffic.iterrows():
    city_count = 0
    road_segment_coord = (row1['y'], row1['x'])

    for index2, row2 in nc_cities.iterrows():
```

Suggestions for improvement:

- Consider vectorizing the distance calculation for efficiency, as nested loops are slow for large datasets.
- Add error handling for missing or malformed coordinates.
- Ensure that the coordinate order (**y, x**) matches geopy's expected (**latitude, longitude**) format.

Py SAS Content: /Public/avo/InnovateDemo3/CitiesPerSegme... x

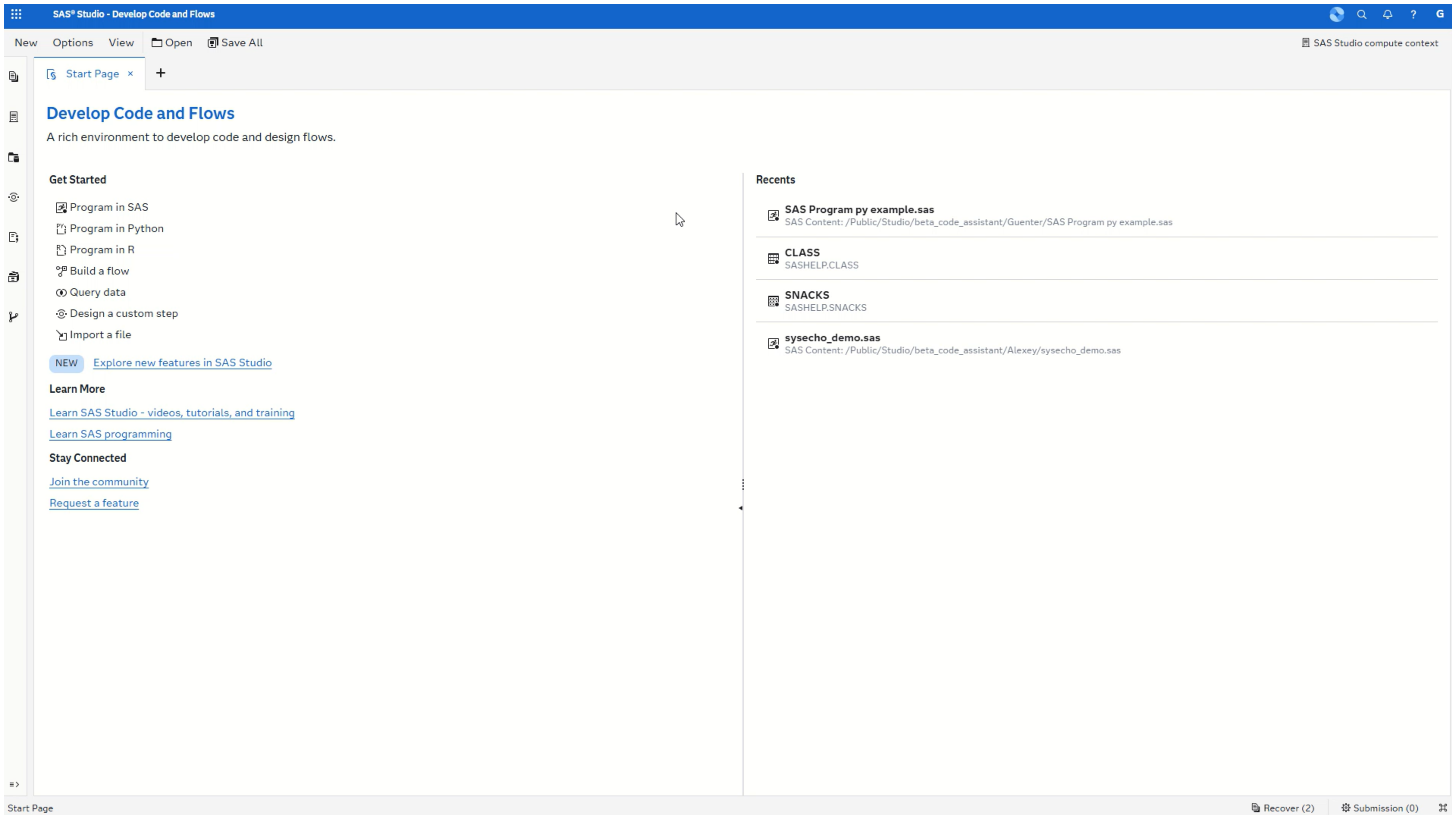
Ask SAS Viya Copilot

AI-generated content. Evaluate accuracy and suitability before use.

Nutzung über Chatbot-Fenster

Code Assistant

Formatieren und kommentieren des SAS-Programms
Python Programm erstellen





Develop Code and Flows

A rich environment to develop code and design flows.

Get Started

-  Program in SAS
-  Program in Python
-  Program in R
-  Build a flow
-  Query data
-  Design a custom step
-  Import a file

NEW [Explore new features in SAS Studio](#)

Learn More

- [Learn SAS Studio - videos, tutorials, and training](#)
- [Learn SAS programming](#)

Stay Connected

- [Join the community](#)
- [Request a feature](#)

Recents

-  **SAS Program py example.sas**
SAS Content: /Public/Studio/beta_code_assistant/Guenter/SAS Program py example.sas
-  **CLASS**
SASHELP.CLASS
-  **SNACKS**
SASHELP.SNACKS
-  **sysecho_demo.sas**
SAS Content: /Public/Studio/beta_code_assistant/Alexey/sysecho_demo.sas

Comes later

Flow Assistant

Überblick

Eine Reihe von „Actions“ zum Erstellen neuer Flows, zur Dokumentation, zur Änderung bestehender Flows und mehr

Flow Assistant

Vom Copilot generierte neue Schritte

Anwendungsfall

Input

Roh-Tabellen mit
Kunden-Churn-Daten

Comes later (Demo)

Task

Vorbereiten der Daten für die Analyse

- ✓ Einhaltung der bestehenden DWH-Standards für Kleinbuchstaben-Spaltennamen
- ✓ Berücksichtigung der PII protections Regeln
- ✓ Die Risikokategorie eines Kunden wird anhand von Geschäftsregeln berechnet.

Vom Copilot generierte neue Steps

The screenshot displays the SAS Data and AI Studio interface. At the top, there's a menu bar with 'New', 'Options', 'View', 'Open', and 'Save All'. Below it, a toolbar contains various icons for running, canceling, and managing the workflow. The main workspace shows a flow diagram with nodes: 'CUST_MAIN' and 'CUST_CARD' leading to 'Lowercase Columns and...', which leads to 'Mask Email Username', then 'Recode Age to Ranges', followed by 'Join All Customer Tables', 'Calculate Risk Category', and finally 'WORK.CUST_DM'. A sidebar on the right provides instructions for building and updating flow steps.

SAS Viya Copilot

and connections for these new nodes.

Steps to mask email, recode age, and write output added.

Join all tables and in the next step calculate risk category using manage columns using the following business rules and load results to CUST_DM table in the WORK library.
Rules for calculating risk category are the following, use them in order of priority:
High -> (income_category = "Less than \$40K" AND months_inactive_12_mon >= 3) OR (income_category = "\$40K - \$60K" AND months_inactive_12_mon >= 5)
Medium -> (income_category = "Less than \$40K" AND months_inactive_12_mon < 3 AND total_trans_ct in (30, 100)) OR (income_category in ("\$80K - \$120K", "\$120K +") AND months_inactive_12_mon >= 3)
Low: all other cases

Building and updating flow steps

- Query: Join All Customer Tables
- Manage Columns: Calculate Risk Category
- Load Table: WORK.CUST_DM

Three nodes added: join tables, calculate risk category, and load to WORK.CUST_DM. Updating flow parameters and connections for these steps.

All tables joined, risk category calculated, and loaded to WORK.CUST_DM.

Source	Name	Label	Type	Length	Format	Informat
Credit_Limit	Credit_Limit		Numeric	8	BEST12.	BEST32.
Total_Revolving_Bal	Total_Revolving_Bal		Numeric	8	BEST12.	BEST32.
Avg_Open_To_Buy	Avg_Open_To_Buy		Numeric	8	BEST12.	BEST32.
Total_Amt_Chng_Q...	Total_Amt_Chng_Q...		Numeric	8	BEST12.	BEST32.
Total_Trans_Amt	Total_Trans_Amt		Numeric	8	BEST12.	BEST32.
Total_Trans_Ct	Total_Trans_Ct		Numeric	8	BEST12.	BEST32.
Total_Ct_Chng_Q4...	Total_Ct_Chng_Q4...		Numeric	8	BEST12.	BEST32.
Avg_Utilization_Ra...	Avg_Utilization_Ra...		Numeric	8	BEST12.	BEST32.
clientnum	clientnum		Numeric	8	BEST12.	BEST32.
attrition_flag	attrition_flag		Numeric	8	BEST12.	BEST32.
customer_age	customer_age		Numeric	8	BEST12.	BEST32.

Comes later (Demo)

Flow Assistant

Vom Copilot erzeugte Beschreibung eines
„Steps“

NewOptionsViewOpenSave All

SAS Data and AI Studio compute context

Start PageFeatureSelection.flw

RunCancel

FlowSubmitted Code and Results

NC_CITIES

Filter out small cities

Drop type of municipality

ReadTraffic.py

Filter out small routes

Get Segment Length

CitiesPerSegment Points.py

Group points to routes

Join charging stations

NC_CHARGING_STAT

Extract stations

temp_dm

Merge into datamart

Scatter Plot

Select a step to view its details.

SAS Viya Copilot

counts, and segment length.

7. Extract Charging Station Counts:

Aggregates charging station counts per route.

8. Join Data:

Merges route segment characteristics with charging station counts.

9. Build Datamart:

Final table is prepared for visualization (e.g., scatter plot of traffic vs. charging stations).

Visualization Step

Scatter Plot:

Plots average daily traffic (X axis) vs. number of charging stations (Y axis), grouped by route ID, enabling analysis of correlations.

Governance and Compliance Notes

PII:

No personally identifiable information is processed; all data is at the route or city level.

Data Lineage:

Each step is traceable, with clear input/output mapping.

Suggestions:

Consider documenting source file versions and update frequency for reproducibility. Ensure spatial joins and aggregations are validated for accuracy.

Let me know if you need a step-by-step breakdown or detailed column mapping!

Current file

Ask SAS Viya Copilot

AI-generated content. Evaluate accuracy and suitability before use.

Summary



Included

- Dokumentation Q&A
- Code Assistant für SAS Code
- Code Assistant für Python Code



Comes next

- Code Assistant für R Code
- Flow Assistant: Steps Generation
- Flow Assistant: Step Description Generation
- Flow Assistant: low Dokumentation
- Und mehr

Fragen?

Guenter.Greulich@sas.com

